

DevOps

S.no	Phases	Tools
1	Building Blocks	Linux, Servers, Basic Networking
2	Plan & Code	Git
3	Release	Jenkins
4	Deploy	Docker & Cloud Computing
5	Operate	OpenShift , Ansible, Terraforms
6	Monitor	Nagios ,Prometheus, Grafana

Linux - Redhat Certified System Administrator (RHCSA)

- Install Red Hat Enterprise Linux
- Get started with Red Hat Enterprise Linux
- Access the command line
- Manage files from the command line
- Get help in Red Hat Enterprise Linux
- Create, view, and edit text files
- Manage local users and groups
- Control access to files
- Monitor and manage Linux processes
- Control services and daemons
- Configure and secure SSH
- Analyze and store logs
- Manage networking
- Install and update software packages
- Access Linux files systems
- Improve command line productivity
- Schedule future tasks
- Analyze and Store Logs
- Archive and Transfer Files
- Manage SELinux security
- Manage logical volumes
- Access network-attached storage
- Control the boot process
- Manage network security

Server

- Manage DNS and DNS servers
- Configure MariaDB SQL databases
- Configure web servers
- Provide file-based network storage
- Access block-based network storage
- High Availability and Load Balancing using HA proxy

Introduction to DevOps

- What is Software Development?

- Software Development Life Cycle
- Traditional Models for SDLC
- Why DevOps?
- What is DevOps?
- DevOps Lifecycle
- DevOps Tools

Version Control with Git

- What is Version Control?
- Types of Version Control System
- Introduction to Github
- Introduction to Git
- Git Lifecycle
- Common Git Commands
- Working with Branches in Git
- Merging Branches
- Git Workflow

Ansible

- Introduction to Ansible
- Deploy Ansible
- Implementing Playbooks
- Managing variable and facts
- Implementing task control
- Deploying files to managed hosts
- Ansible Modules
- Managing large projects.
- Ansible Roles
- Automating Linux administration tasks

Continuous Integration with Jenkins

- Introduction to Continuous Integration
- Jenkins Master-Slave Architecture
- Understanding CI/CD Pipelines
- Configuring Jobs
- Managing Plugins
- Configure the CI/CD pipelines.
- Role based access control
- Creating an end-to-end automated CI/CD Pipeline
- High Availability of Jenkins (Master-Slave Configuration)

Containerization Using Docker

- Introduction to Docker
- Understanding Docker Lifecycle
- Components of Docker Ecosystem
- Common Docker Operations
- Creating a Docker Hub Account
- Committing changes in a Container
- Pushing a Container Image to docker Hub
- Creating Custom Docker Images using Docker file

AWS Cloud

- Cloud Computing & AWS
 - Cloud Computing(Service & Deployment Model) and Benefits of Cloud Computing
 - How to Create AWS Account?
- AWS Global Infrastructure
 - Global Infrastructure of AWS
 - Unpacking the Concept of Availability Zones
 - Shedding Light on Edge Locations in the AWS Network
- Compute in the Cloud
 - Amazon Elastic Compute Cloud (Amazon EC2) and Instance Types
 - Amazon EC2 Pricing
 - Amazon EC2 Auto Scaling
 - Elastic Load Balancing
 - AWS Container Services
- Account Security in AWS
 - AWS Identity and Access Management (IAM)
 - AWS Application Security (AWS WAF, AWS Shield)
- Storage and Databases in AWS
 - AWS Storage (Block, File, and Object Storage)
 - AWS Simple Storage Service (S3) and S3 Bucket Policy
 - AWS Database (RDS)
- Networking on AWS
 - IP Addressing and Classless Interdomain Routing
 - AWS Virtual Private Cloud(VPC)
 - AWS VPC Security(NACL and SG)
 - Route 53
 - VPN and Direct Connect
- Monitoring and Analytics
 - Amazon CloudWatch
 - AWS CloudTrail and AWS Trusted
- Pricing and Support by AWS
 - AWS Pricing Model
 - AWS pricing tools(Cost explorer and Budgets)
 - AWS Support plans and AWS Marketplace

Terraform Modules & Workspaces

- What is Infrastructure as a code
- IaC vs Configuration Management
- Introduction to Terraform
- Installing Terraform on AWS
- Basic Operations in Terraform
 - init
 - Plan
 - Apply
 - Destroy
- Terraform Code Basics
- Deploying an end-to-end architecture on AWS using Terraform

Red Hat OpenShift Administration with Kubernetes

- Introduction to Kubernetes and OpenShift
- Kubernetes and OpenShift Command-Line Interfaces and APIs
- Run Applications as Containers and Pods
- Deploy Managed and Networked Applications on Kubernetes
- Manage Storage for Application Configuration and Data
- Configure applications to work with Kubernetes for high availability and resilience.
- Manage Application Updates

Monitoring

- Intro to Monitoring Best Practices
 - white-box vs blackbox metrics
 - Push vs Pull data collection
 - SLIs and SLOs
- Prometheus Overview and Architecture
- Prometheus Installation & Configuration
- Metrics
 - Prometheus data model
 - Metric types
 - Labels
- System and platform monitoring with Prometheus.
 - Jobs and Instances
 - Service Discovery
- Application Monitoring
- Querying (PromQL)
- Alerting
- Operating Prometheus
- Security
- Integrations
- Visualizing Your Data (with Grafana)
- Installing Grafana
- Configuring Prometheus as a data source
- Creating Dashboards

Some Project Devops

1. Automated CI/CD Pipeline with Jenkins and AWS:

- Description: Build a CI/CD pipeline using Jenkins for an application hosted on an AWS instance. The application should be tested, built, and deployed automatically every time there is a new commit.

- Components:
 - Jenkins: Set up a Jenkins job that watches your GitHub repository for changes.
 - AWS: Use an EC2 instance as your build server and host your application on an EC2 instance.
 - Linux: Install Jenkins on a Linux EC2 instance and configure your application to run on a Linux server.
 - Docker: Create Docker images for your application.
 - Kubernetes: Optionally, deploy your Docker images to a Kubernetes cluster hosted on AKS.

2. Infrastructure as Code (IaC) with Terraform and AWS:

- Description: Create an automated infrastructure deployment using Terraform and Jenkins to provision AWS resources such as EC2 instances, S3 buckets, and RDS databases.
- Components:
 - Jenkins: Automate Terraform script execution using Jenkins jobs.
 - AWS: Use Terraform to manage AWS resources.
 - Linux: Develop and execute Terraform scripts on a Linux machine.
 - Docker: Use Docker for running Terraform scripts in a consistent environment.
 - Kubernetes: Optionally, use Kubernetes for provisioning clusters and managing infrastructure on AWS.

3. Dockerized Microservices Application Deployment:

- Description: Develop a microservices application with Docker containers and deploy them on a Kubernetes cluster.
- Components:
 - Jenkins: Create a CI/CD pipeline to build, test, and deploy your microservices application.
 - AWS: Host the Kubernetes cluster using AWS EKS.
 - Linux: Use Linux-based Docker images for your microservices.
 - Docker: Develop each microservice as a Docker container.
 - Kubernetes: Deploy your microservices application to a Kubernetes cluster.

4. Serverless Application Deployment:

- Description: Create a serverless application using AWS Lambda and deploy it using Jenkins.
- Components:
 - Jenkins: Automate the deployment of your serverless application using Jenkins jobs.
 - AWS: Host the serverless application using AWS Lambda, API Gateway, and other services.
 - Linux: Use a Linux machine for development and Jenkins server.
 - Docker: Use Docker to package serverless functions for easy deployment and testing.
 - Kubernetes: Integrate serverless functions with Kubernetes for hybrid workloads.

5. Monitoring and Logging with Prometheus, Grafana, and EFK:

- Description: Set up a monitoring and logging system for an application running on a Kubernetes cluster using Prometheus, Grafana, and the Elastic (EFK) stack.
- Components:

- Jenkins: Automate the deployment of monitoring and logging components on your Kubernetes cluster.
- AWS: Host your Kubernetes cluster on AKS.
- Linux: Use Linux-based containers for Prometheus, Grafana, and EFK stack.
- Docker: Deploy Docker containers for each monitoring and logging tool.
- Kubernetes: Manage monitoring and logging components on the Kubernetes cluster.